

Particle Swarm Optimization Algorithm Acceleration with a Simple 5-Stage Teaching Processor

Andrea Ortenzi, Valentina Moretti, Roberto Neglia, Johannes Michaelis, Anatole Vercelloni

August 21, 2024

1 Algorithm

1.1 Particle Swarm Optimization

The Particle Swarm Optimization (PSO) is an algorithm that solves an optimization problem by a distributed search in space. A population of particles is distributed in the domain and iteratively repositioned according to their position, velocity and both the best known position of themselves and the entire pool of particles.

The idea is taken from swarm intelligence, which describes the natural process of individuals improving by themselves while considering gathered findings of the swarm.

The PSO algorithm is suited for optimization problems of large domains, as it employs many starting points in the search area. However the algorithm uses a metaheuristic approach and thus can not guarantee that an optimal solution is found. Particle Swarm Optimization does not rely on gradients and thus is especially suited for non-differentiable problems.

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be the function that is to be minimized.

Furthermore, let ∇f not be defined, s.th. gradient-based algorithms like the CG algorithm can not be applied.

The goal is to find the vector $\mathbf{x}$ for which the function f is minimal.

As shown in [Figure 1](#) (a), particles are initially distributed following a normal distribution in space, and given an initial random velocity. The swarm's best known position is evaluated by finding the particle with the minimal objective function's result after the random initialization. In every iteration, each particle is updated by assigning a new velocity. The new velocity equals a weighted sum of its current velocity, a randomly weighted (ϕ_g) difference between its current position and its own best known position and a randomly weighted (ϕ_s) difference between its current position and the swarm's best known position. The particle's position is then updated according to its current position and the assigned velocity. After updating all particles, the function is evaluated at each particle's position and the position of the one with the minimal result is set to be the new group's best known position. This iterative process is stopped after a predefined convergence criterion is met.

The sequence in [Figure 1](#) shows the convergence of all particles to the objective function's minimum after 0, 15 and 30 iterations.

2 Implementation

2.1 PSO algorithm

The algorithm is implemented in both C and C++. As compiling in Cudasip Studio was restricted to C programs only, the C code is described here.

The task is to optimize two functions mapping from $\mathbb{R}^2$ to $\mathbb{R}$, namely

$$z = x^2 + y^2 \tag{1}$$

$$z = 0.26 \cdot (x^2 + y^2) - 0.48 \cdot x \cdot y \tag{2}$$

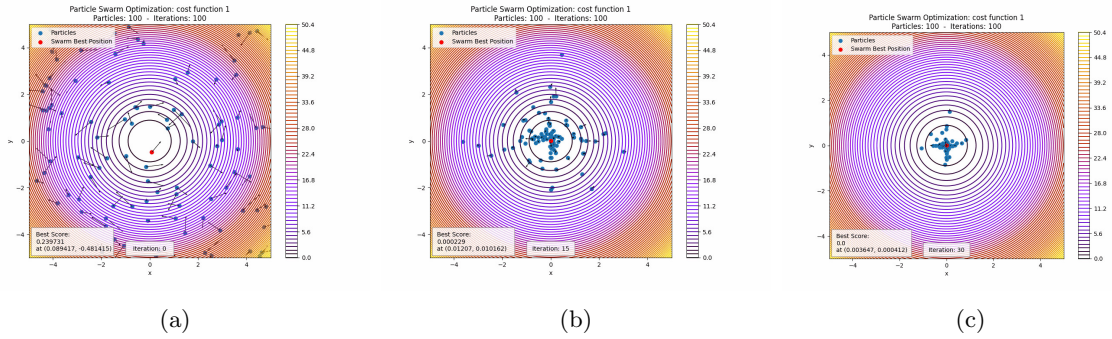


Figure 1: Visualization of the objective function and the particles after 0, 15 and 30 iterations.

The program takes the following parameters as command line arguments, if the `DEBUG` flag is set to false.

- `n_particles`: amount of particles (default: 100)
- `n_iterations`: number of iterations (default: 100)
- `b_lo`: lower bound for domain (default: -5)
- `b_up`: upper bound for domain (default: 5)

The C program provides two `structs`, one for describing particles and one for describing the swarm. The `psoSwarm` struct holds the position of the particle with the best score of the swarm and the best score itself. The `psoParticle` struct holds variables for a particle's position, velocity, the best position found so far and the corresponding minimal value. After allocating memory and initializing the swarm and all the particle instances, the particles are assigned a random position sampled from a uniform distribution. The objective function is evaluated at the position and the result is stored in the struct's variable `best_pos`. Furthermore, the swarm's best position is evaluated and set. After initialization, the particles can be iteratively updated by applying the procedure described in [section 1](#).

The program provides some optional output of the ongoing computation while iterating. This can be used to evaluate the computation in detail, but mostly it is useful for visualization the ongoing optimization. The source code also provides a Python script which generates an animated gif-file depicting the particles position and velocity updates over time. After the given iterations, the program stops and displays the best position and its objective function's value.

For both functions (1) and (2), the optimal position is (0,0) with $f(0,0) = 0$.

2.2 Fixed-point Arithmetic

Fixed-point arithmetic allows representing fractional numbers as integer numbers with an implicit scaling. In computation it is sometimes worth trading memory and computation complexity for a less exact result. In the s15.16 presentation, an integer value is taken which consists of 32 bits. The first bit represents the sign, the following 15 bits represent the integer part of the number and the remaining 16 bits represent the fractional part of the number. This implies a bit shift of 16 bits for the number when comparing it to a floating-point datatype of the same size (float: 4 bytes/32 bits).

Precisely, in our implementation, to convert a double value (8 bytes) to an integer value (4 bytes), we can multiply the double value with a bitshifted 1 and cast it to a 32bit-integer:

```
#define FRAC.BITS 16
...
int32_t fp_from_double(double a) {
```

```

    return (int32_t)(a * (1 << FRAC_BITS));
}

```

Listing 1: Converting a double to a fixed-point number

Vice versa, a fixed-point number can be converted into a double representation by dividing by the shifted value:

```

double fp_to_double(int32_t a) {
    return ((double)a / (1 << FRAC_BITS));
}

```

Listing 2: Converting a fixed-point to a double number

Addition and subtraction of fixed-point numbers can be performed without any further consideration, however multiplication and division require shifting of one operand:

```

int32_t fp_mul(int32_t a, int32_t b) {
    return (int32_t)(((long long)a * (long long)b) >> FRAC_BITS);
}

```

Listing 3: Multiplication requires a shift to the right by FRAC_BITS

The casting to `long long` is done to perform the multiplication of the two fixed-point numbers at sufficient precision before shifting and casting the result back to a `int32_t`-representation.

```

int32_t fp_div(int32_t a, int32_t b) {
    return (int32_t)((((long long)a) << FRAC_BITS) / (long long)b);
}

```

Listing 4: Division requires a shift to the left by FRAC_BITS

For the existing implementation, the datatypes just had to be adjusted to be `int32_t` instead of `double` and the previously described functions had to be used for multiplication and division. The conversion of floating-point to fixed-point numbers is only required when initializing the values and for converting them back for a final presentation of the calculated results.

Random number generation requires some additional attention, we decided to generate the random number as a floating-point number, convert it to a fixed-point number and map it on the given interval. It's worth noticing that the multiplication once again requires a bitshift of one operand. This could have been performed by using the `fp_mul`-function, but we decided to write it here explicitly:

```

int32_t fp_rnd(int32_t v_lo, int32_t v_up) {
    // Generate a random number between 0 and 1
    double random_double = (double)rand() / (double)RAND_MAX;
    // Scale the random number to fit within the desired range [v_lo, v_up)
    int32_t random_fixed_point = fp_from_double(random_double) * ((v_up - v_lo) >>
        FRAC_BITS) + v_lo;
    return random_fixed_point;
}

```

Listing 5: Generating a random fixed-point number between `v_lo` and `v_up`

For problem (1), the result is position (239, 27) in fixed-point representation, being (0.003647, 0.000412) in floating-point.

Furthermore, the result of the objective function is 0 in fixed-point representation and thus 0 in floating-point.

The result is thus correct, however the position contains some round-off errors, which is obvious as the values the algorithm calculates with are taken off a fair amount of precision compared to using `double`.

3 Codasip

3.1 Compilation

3.2 Simulation

The total number of clock cycles to complete the program is 456696604 (IA) .

3.3 Profiling

We report the profiling results for the IA model in the Figure 2 and of the CA model in the Figure ??.

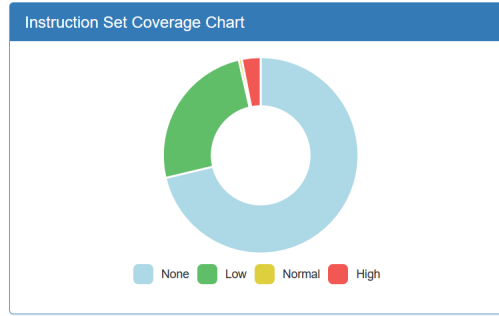


Figure 2: Profiling with the IA model

Top by Usage		
Instruction	Count	Percent
srl xpr_general , xpr_general , <imm>	56774464	14 %
add xpr_general , xpr_general , xpr_general	56622057	13.9 %
and xpr_general , xpr_general , xpr_general	53996665	13.3 %
slli xpr_general , xpr_general , <imm>	51491988	12.7 %
bitu x0 , xpr_general , <imm>	50039774	12.3 %

Figure 3: Profiling with the IA model: instruction set sorted by top usage

3.4 Analysis of the results

Our objective is to reduce the number of clock cycles to complete the program. One way to do this is to identify the most frequently used instructions that can be removed from the program. We analyze the profiling results according to the following criteria:

- We are looking for instructions that are used frequently.
- We are looking for instructions that don't require any additional resources. Indeed we are using a processor with 2 read ports (src1, src2) and one write port (wrt0) and we don't want to add any additional hardware resources. The decision of adding a resource must consider a trade off between increased performance versus increased chip cost and power consumption.
- Since we are not adding ant write port, we must identify instructions that don't write to multiple register locations. Inspecting the instructions, we consider that branch instructions do not write to the register file and that this notation {df#} is used to indicate that the instruction writes to the same register location.
- Since the processor has only a single immediate value generator, we are looking for pairs of instructions that don't require two immediate values.

- We consider also the number of operands going into the ALU. The processor has two operands that can be either two registers or a register and an immediate. Consequently, we are looking for pair of instructions that don't require more than two operands, apart from the x0 register which is automatically set to 0. We take into account also the fact that intermediate values doesn't require to be read out of the Register File or to be generated as Immediate Values.

3.5 Instruction replacement

The removable instructions are:

- ```
andi xpr_general{df0} , xpr_general , <imm>
sub xpr_general{df0} , x0 , xpr_general{df0}
it is used 10.9%, counts: 49794986
```
- ```
sub xpr_general{df0} , x0 , xpr_general{df0}
and xpr_general{df0} , xpr_general , xpr_general{df0}
it is used 10.9%, counts: 49794986
```

We notice that the two instructions can be replaced by a single instruction thanks to the fact that they use the same register location in each pair. Moreover, both pairs require one immediate value at maximum and have only two operands (apart from the x0 register and the intermediate value). We also notice that has an high count corresponding to the 10.9%, which is a significant amount. Consequently, these pairs are fulfilling all the criteria we have defined in the previous section.

The reason why we don't consider the other instructions is that it "overlaps" with the other instructions. Indeed, once we substitute the first pair of instructions, the other pair doesn't appear no longer. A possible explanation is that the other pair of instructions in someway concatenated with the first pair of instructions.

We decide to replace the first pair of instructions following the arithmetic equation below:

```
rdandi = src1 & imm
rd = 0 - rdandi
```

That can be rewritten as:

```
rd = 0 - (src1 & imm)
```

The new instruction can be modeled as an immediate instruction, being the negative value of an AND immediate operation. This instruction fell into the category of the n I-Type, immediate instructions.

3.6 Implementation of the optimization

In the previous section we have defined the new instruction. Now we need to implement it defining the instruction operands, its equation and whether a result are to be stored into memory or the register file.

In order to enable the c-compiler, assembler and simulator to use the new instruction, we need to define the new opcode in the ISA model. From the riscv doc page 147, we find the opcode for the new instruction (Figure 4). We build the opcode using the table and we obtain:

```
Inst[1:0] = 11 (This indicates the instruction type, specifically an OP-IMM instruction.)
Inst[4:2] = 100 (This specifies the column information.)
Inst[6:5] = 00 (This specifies the row information.)
```

Gathering them together we obtain:

```
Inst[6:0] = 00 100 11
```

The last step to complete the opcode, in order to match the ITYPE instruction format, is to add the func3 bit code. Since there are no other instructions with the same opcode we can choose as func3 bit code for our instruction any value between 0b000 and 0b111. We choose 0b000. The

inst[4:2] inst[6:5]	000	001	010	011	100	101	110	111 ($> 32b$)
00	LOAD	LOAD-FP	<i>custom-0</i>	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	<i>custom-1</i>	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	<i>reserved</i>	<i>custom-2/rv128</i>	48b
11	BRANCH	JALR	<i>reserved</i>	JAL	SYSTEM	<i>reserved</i>	<i>custom-3/rv128</i>	$\geq 80b$

Table 24.1: RISC-V base opcode map, inst[1:0]=11

Figure 4: From riscv doc page 147, finding the opcode for the new instruction

final opcode is 0b0000001011. We need to add the new instruction "iandi" to the ITYPE_OPCODES enums inside /model/share/include/opcode.hcodal file since our instruction is a itype. We've called it ITYPE_IANDI since it's simply the inverse of andi.

We now need to add the new instructions to the compiler and assembler. We define the new instruction mnemonics using the following DEF_OPC macro.

```
DEF_OPC(iandi, "iandi", ITYPE_IANDI)
```

This macro creates the element name of opc_ia for the instruction opc_iandi. The "iandi" instruction will be used in the assembler with a syntax similar to: iandi x5, x6, 0xff. The full opcode is specified as the third argument, and it's already defined in the enum.

The new instruction "iandi" needs to be added to the set of instructions categorized under "opc_type_alu". Then, the function of the new instruction needs to be defined in the switch statement inside the function "i_type_alu" in the file "isa.codal" in this way:

```
case ITYPE_IANDI:
    result = - (src1 & immediate);
    break;
```

3.7 Use of the new instruction

To use the new instruction we need to regenerate the SDK(ia) model from Workflow. We notice it takes a long time to regenerate the SDK(ia) model. Then, we clean compilation from Workflow. Also if there is no need to assign again the SDK, we do it for the sake of completeness. We compile the program with the new instruction. Finally, we do the profiling.

3.8 Analysis of the results after the optimization

The total number of clock cycles to complete the program is now 406901774 (IA). We notice we have obtained a reduction of 49794830 clock cycles.

4 Conclusions

To conclude, we can say that we were able to implement the PSO algorithm and get good results. Then we managed to understand how codasip work, we were able to reduce the number of clock cycles by removing instructions that are the most frequently used. We compiled the IA model and profiled it to observe this improvement. However, we also tried to do the same analysis with the CA model but even if the compilation worked, we did not figured out why but the profiling did not work.